MBEYA UNIVERSITY OF SCIENCE AND TECHNOLOGY

<u>GROUP ASSIGNMENT</u>

DERPAMENT: **COMPUTER SCIENCE AND AND ENGINEERING**

COURSE: **COMPUTER SCIENCE (3RD YEAR).**

MODULE NAME: **APPLICATION DEVELOPMENT FOR MOBILE DEVICES**

MODULE CODE: **CS 8306**

FACILITATOR: **Mr. AMANI**

RUKOIJO GROUP NAME: **CODEWISE**

GROUP MEMBERS:

NO	NAME	CA NUMBER
1.	CLEVEN SAMWEL SWAI	CA/BCS/25/12373
2.	FRANK NTARO JOSEHAT	CA/BCS/25/2748
3.	SAMWEL DEOGRAS CHAMBILO	CA/BCS/25/8760
4.	UPENDO JOHN MSOKE	CA/BCS/25/4530
5.	TUMLACK OSIA MPANDILA	CA/BCS/25/8762

FINAL REPORT: DEBTAPP MANAGEMENT SYSTEM

1. Introduction

Problem description

- Small and medium-sized retail businesses often rely on traditional paper-ledgers to track customer credit (debts). This manual system is prone to physical damage, loss, and lack of transparency. Customers often lose track of what they owe, and shopkeepers struggle to manage records across multiple branches or shops.

Motivation

- The motivation behind DebtApp is to digitize the trust-based credit system common in local communities. By providing a "source of truth" accessible to both parties, we reduce financial disputes and improve the debt recovery process through organized digital records.

PROJECT OBJECTIVES

Main objective.

To develop a web application for debt management.

Specific objectives.

i. To develop login and registration form for both shopkeeper and customer.
ii. To develop shopkeeper and customer dashboard.
iii. To integrate the developed frontend components with backend (Node.js + Express.js) via RESTFul APIs.
iv. To develop database to store and retrieve customer and shopkeeper data.
v. To connect backend with mysql database

2. System requirements

Functional requirements

- User roles: Distinct dashboards for Shopkeepers (management) and Customers (view-only).
- Shop management: CRUD (Create, Read, Update, Delete) functionality for shops.

- Debt recording: Shopkeepers can add debts using a customer’s phone number.
- Payment processing: Ability to mark debts as paid or update partial payments.
- Registration lock: Customers can only register if their phone number exists in the database.
- The system must be installable on mobile devices via PWA technology.

Non-functional requirements

- Security: Password hashing using industry-standard algorithms.
- Usability: A mobile-responsive UI built with Tailwind CSS for use on tablets and phones.
- Offline capability: Use Service Workers to cache assets.
- Performance: Fast API response times using Node.js asynchronous non-blocking I/O.
- Reliability: Use of Foreign Key constraints to ensure data integrity.

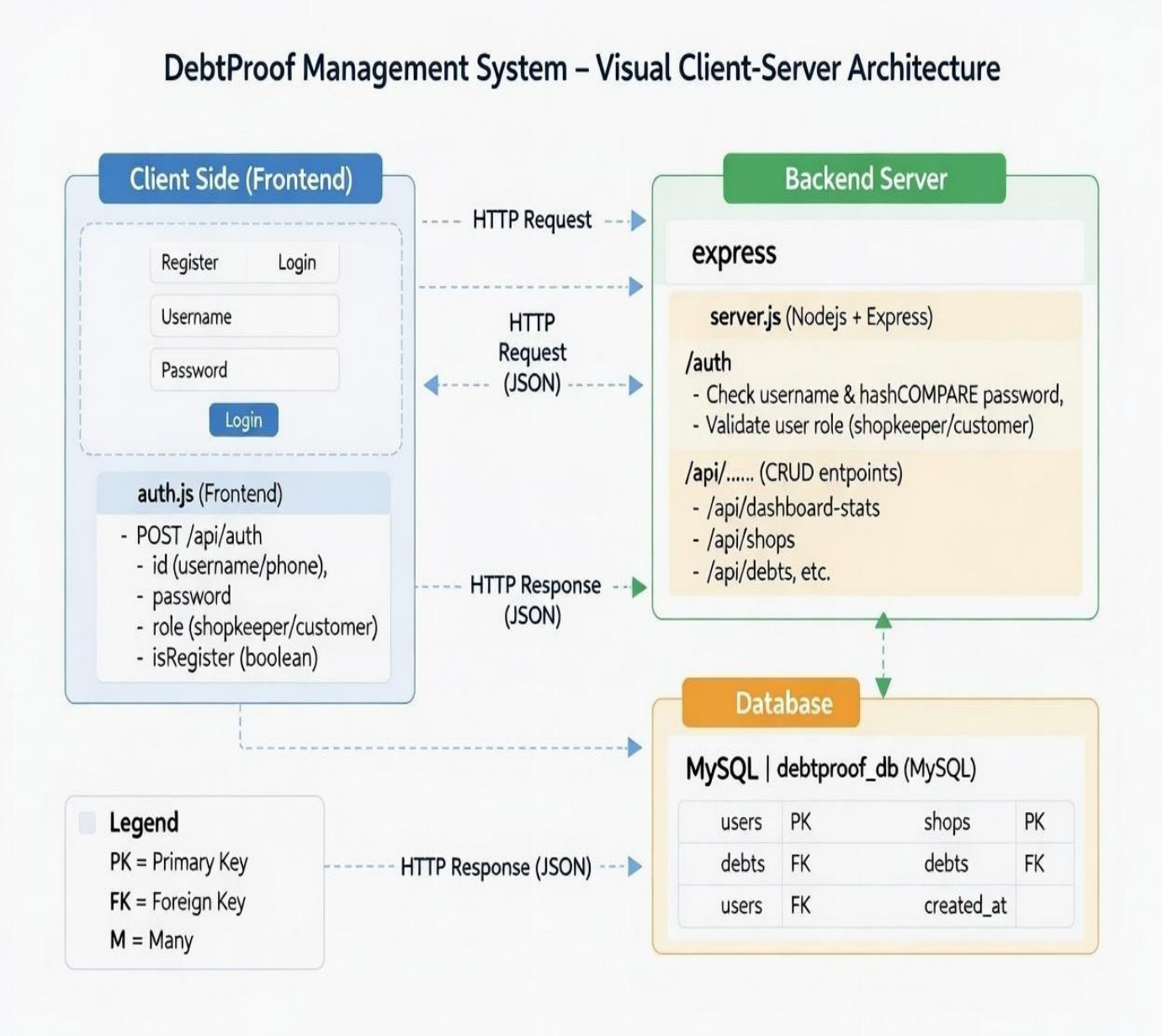
3. System Architecture

Overall system design

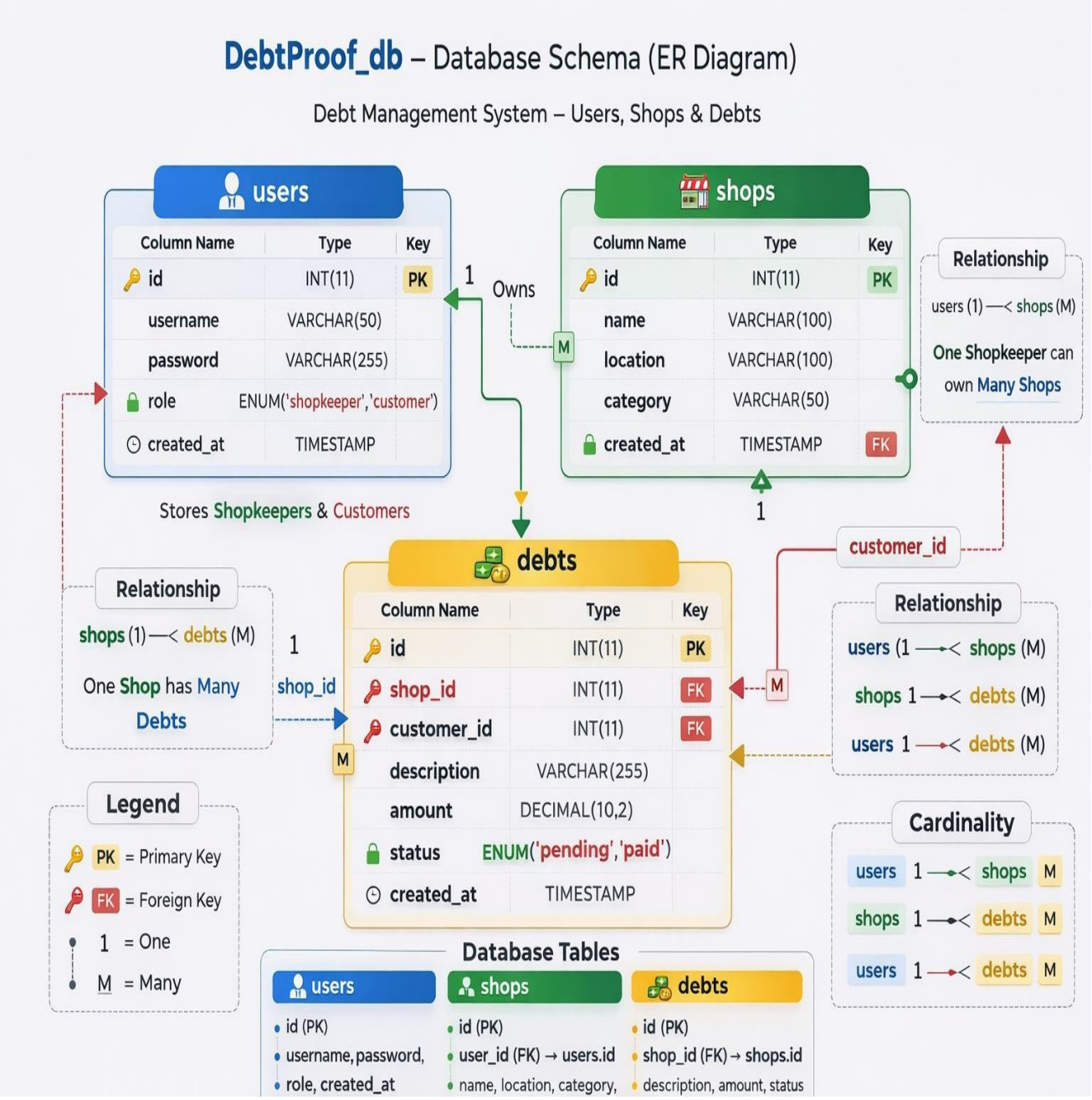
The system follows a Monolithic Web Architecture where the Node.js server serves both the static frontend files and the REST API endpoints, communicating with a centralized MySQL database.

Client-Server Architecture Diagram

The frontend sends HTTP requests (GET, POST, PUT, DELETE) via the Fetch API. The Node.js server processes these requests, verifies authentication, and interacts with the MySQL database to return JSON data.


DebtProof Management System – Visual Client-Server Architecture
Client Side (Frontend)
Register
Login
Username
Password
Login
auth.js (Frontend)
- POST /api/auth
- id (username/phone),
- password
- role (shopkeeper/customer)
- isRegister (boolean)
HTTP Request
HTTP Request (JSON)
HTTP Response (JSON)
Backend Server
express
server.js (Nodejs + Express)
/auth
- Check username & hashCOMPARE password,
- Validate user role (shopkeeper/customer)
/api/..... (CRUD entpoints)
- /api/dashboard-stats
- /api/shops
- /api/debts, etc.
Database
MySQL | debtproof_db (MySQL)
users PK shops PK
debts FK debts FK
users FK created_at
Legend
PK = Primary Key
FK = Foreign Key
M = Many
HTTP Response (JSON)

Database Schema (ERD)



Authentication mechanism

The system uses a Token-based authentication logic. Upon login, a unique session token is generated. On the frontend, localStorage stores the user's role and identity to maintain state across pages.

Security measures used

- Bcrypt Hashing: No plain-text passwords stored.
- SQL Injection Prevention: Use of parameterized queries (? placeholders) in the mysql2 library.
- Access Control: Backend logic checks user roles before allowing access to management routes.

4. Implementation

Details Technologies used

- Frontend: HTML5, Tailwind CSS, Vanilla JavaScript (ES6).
- Backend: Node.js, Express.js.
- Database: MySQL.
- PWA: Service Workers, Web App Manifest.

- Libraries: bcrypt (encryption), mysql2 (database driver), cors (cross-origin resource sharing).

Frontend implementation

The frontend is built as a Single Page Application (SPA) experience using standard JavaScript. Managed UI state (like the sidebar toggle and PWA install button) using native DOM manipulation to keep the app lightweight. A common.js file handles global logic like the sidebar toggle, logout, and the apiFetch wrapper which ensures consistent error handling across the app.

Backend implementation

The REST API is modular. Routes are defined for /api/auth, /api/shops, /api/debts, and /api/profile. The server uses Express middleware to parse JSON bodies and handle static file routing.

Challenges faced

- Registration Logic: Creating a workflow where a user exists (as a placeholder) before they have a password required a specific "Update-on-Register" SQL logic.
- Relational Integrity: Ensuring that deleting a shop also cleared its associated debts without leaving "orphan" records.

Assumptions made

- Users will use their Phone Number as their primary Username.
- A customer only needs to see their debts and cannot edit them.
- Users are using modern browsers (Chrome/Safari) that support Service Workers and the display-mode: standalone media query.

5. Security considerations

How authentication is handled

The /api/auth route handles two modes: isRegister (True/False). If registering, the system validates the "invite" (existence of the phone number). If logging in, it validates the role and password.

How passwords are stored

Passwords undergo a process called salting and hashing.

Hash = bcrypt(password, salt_rounds)

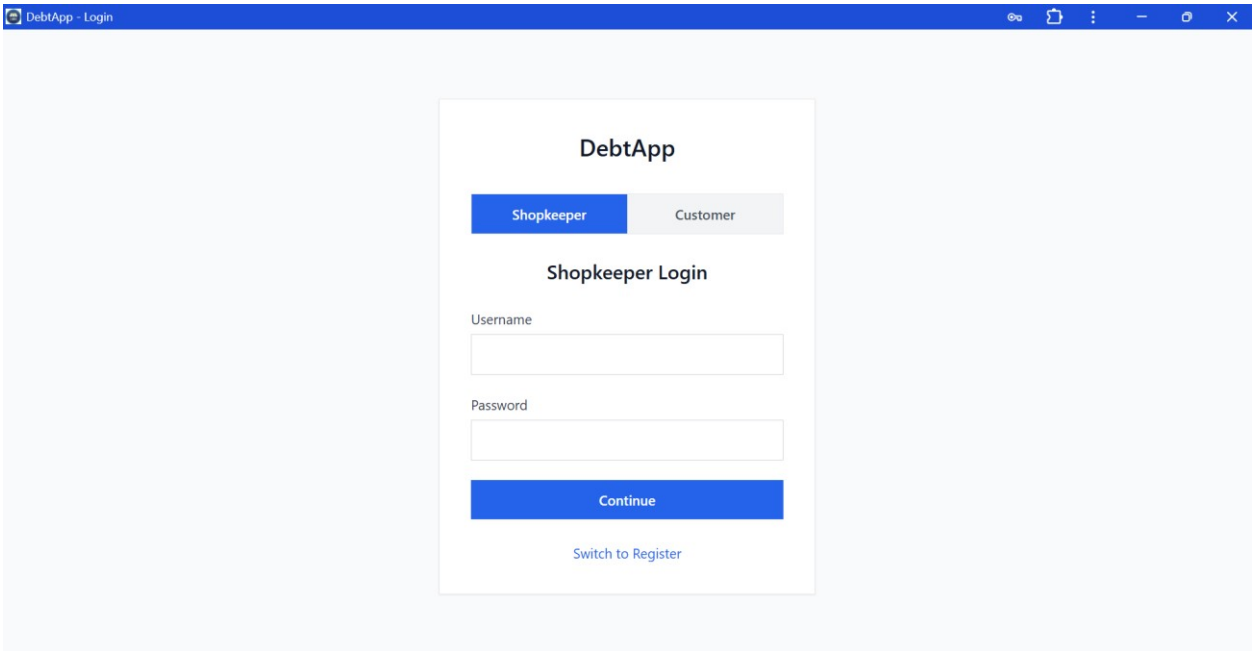
Even if the database is compromised, the actual passwords cannot be reversed.

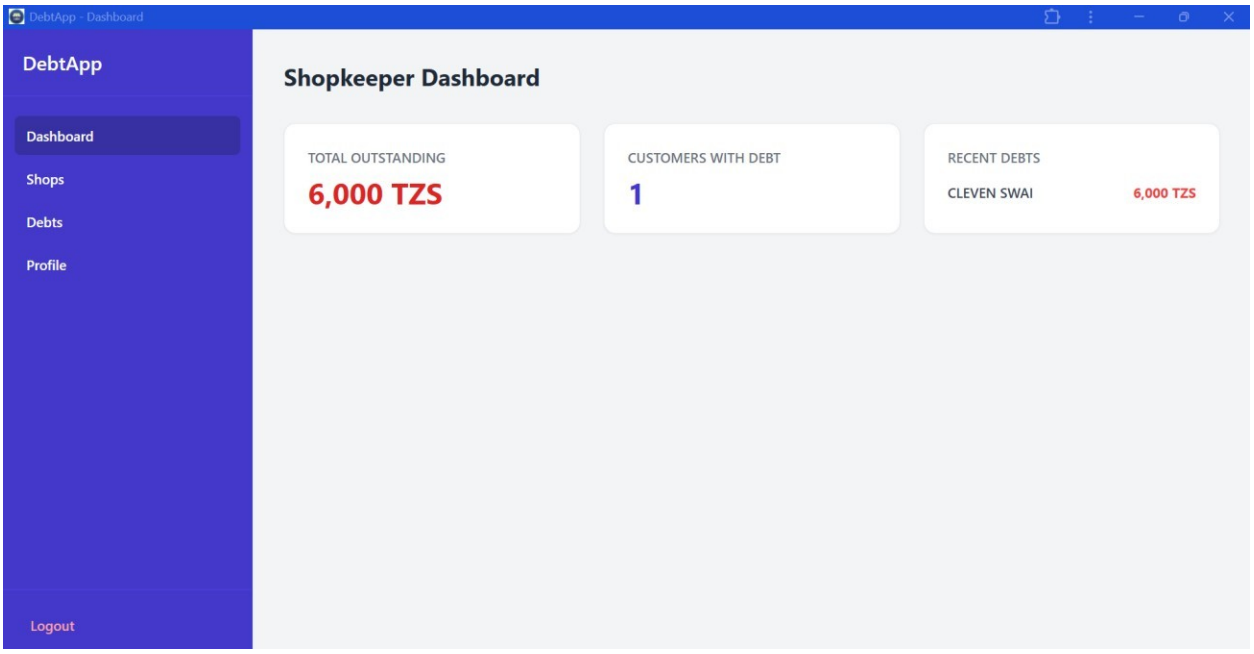
Access control implementation

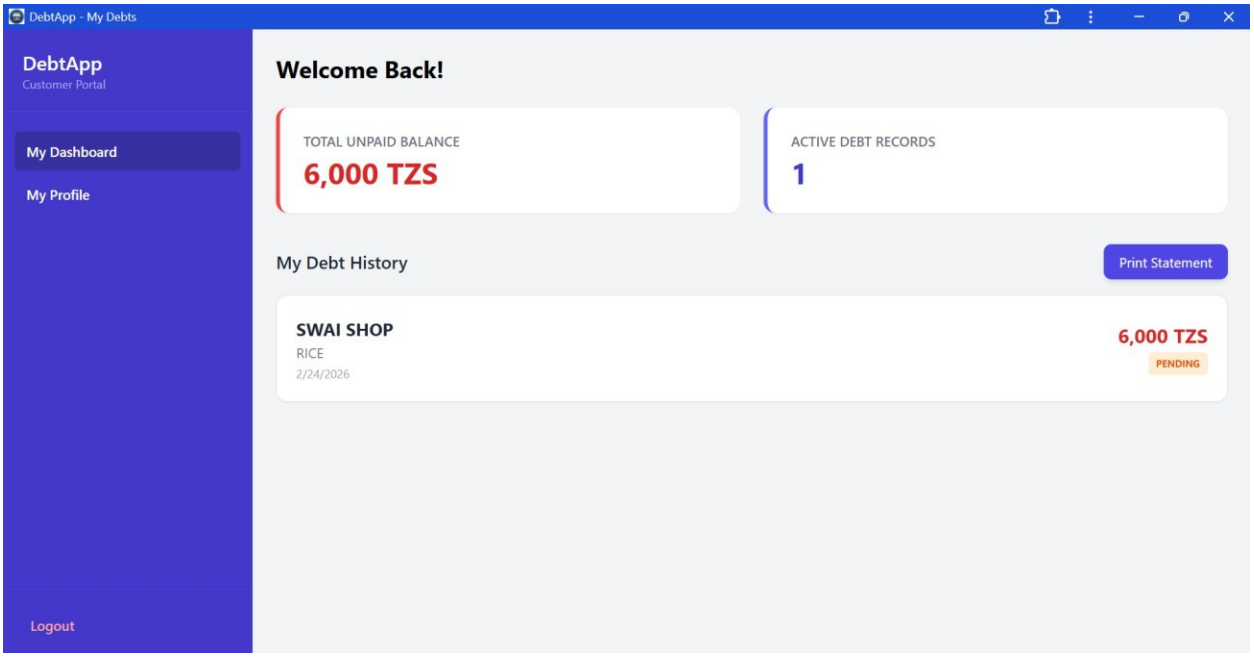
Frontend elements (like the "Shops" menu) are hidden or shown based on the role stored in localStorage. Backend queries for customer data specifically filter by the logged-in user's ID to prevent them from seeing other people's debts.

6. Testing and results

Screenshots of the application




DebtApp - Dashboard
DebtApp
Dashboard
Shops
Debts
Profile
Logout
Shopkeeper Dashboard
TOTAL OUTSTANDING
6,000 TZS
CUSTOMERS WITH DEBT
1
RECENT DEBTS
CLEVEN SWAI
6,000 TZS


DebtApp - My Debts
DebtApp
Customer Portal
My Dashboard
My Profile
Logout
Welcome Back!
TOTAL UNPAID BALANCE
6,000 TZS
ACTIVE DEBT RECORDS
1
My Debt History
Print Statement
SWAI SHOP
RICE
2/24/2026
6,000 TZS
PENDING

Major features working

- The "Invite" flow: Verified that a user cannot register unless a shopkeeper adds them first.
- Debt reduction: The PUT /api/debts/:id/pay endpoint correctly calculates remaining balances or marks them as "paid."
- Multi-shop filtering: Shopkeepers successfully see only the debts belonging to their specific shops.
- PWA installation: The "Install App" button successfully install the app on your device and immediately app launches to the device home screen.
- Sidebar Navigation: Fully responsive toggle menu working on mobile and desktop.
- CRUD operations: Shopkeepers can successfully add and edit shop details with real-time UI updates.

Incomplete features

- Automated SMS notifications: Currently, customers must log in to see debts; future versions will push SMS alerts.
- Password recovery: A manual reset by the admin is required if a password is lost.

7. Future work

Possible improvements

- M-Pesa/payment Integration: Allowing customers to pay their debts directly through the app.
- Detailed analytics: Adding charts to show monthly debt collection trends for shopkeepers.
- Multi-language support (English and Swahili).

Scalability considerations

For higher loads, the system can be migrated to a Cloud database (like AWS RDS) and the backend can be containerized using Docker to handle thousands of concurrent shopkeepers across different regions.

LINK TO GITHUB ACCOUNT.

https://github.com/frankntaro/DEBTS-MANAGEMENT-WEBAPP

LINK TO VIDEO.

https://drive.google.com/file/d/1-zcKmdX1oyuAWBWQ4IUUhjVOwWfSHHZf/view?usp=sharing